Sistema de Bazar Luz da Esperança

Documentação do MVP

PDV com operação offline, entrada e triagem de doações, estoque duplo Bazar ⇄ Doações, controle de caixa, financeiro e prestação de contas por doador e campanha.

Versão 0.2.0 · 05/10/2026 · Aprovado no 2º reteste de QA
Centro Espírita e Obras Sociais · Parnaíba, PI

1. O que é e qual o intuito

O Sistema de Bazar substitui o controle manual do bazar beneficente do **Centro Espírita e Obras Sociais Luz da Esperança** (Parnaíba – PI) por uma aplicação web única que junta:

- **Ponto de venda (PDV)** que funciona mesmo sem internet;
- **entrada e triagem de doações**;
- **dois estoques independentes**, Bazar (o que é vendido) e Doações (o que vai para a assistência social);
- **controle de caixa** por terminal;
- **financeiro** (contas a pagar e fiado);
- **prestação de contas por doador/campanha**.

Problemas que resolve

Hoje (manual)	Com o sistema
Sem visão única do dinheiro e do caixa	Caixa por terminal com abertura, sangria/suprimento e fechamento conferido; resumo financeiro por período
Itens migram do bazar para a assistência sem registro	Transferência entre os dois estoques sempre registrada e auditada (motivo + usuário)
Impossível dizer quanto cada parceiro (ex.: C&A, campanhas) gerou	Toda doação nasce ligada a parceiro/campanha; o relatório mostra recebido, descartado, vendido, receita e o que resta em estoque
Planilhas paralelas e erros de soma	O servidor recalcula tudo; o saldo de estoque é mantido pelo banco e não pode ser editado à mão

Metas do piloto

- Operar **5 dias úteis seguidos** com 100% das vendas e transferências registradas.
- Totais de caixa **batendo com a contagem física**.
- Zero planilhas paralelas. Entrega final: fim de novembro / início de dezembro de 2026.

Sobre ser um SaaS

O MVP é hoje um **monolito single-tenant** (uma instituição por instalação: um banco, uma instituição cadastrada, um conjunto de parâmetros). A arquitetura já facilita evoluir para SaaS — código organizado por domínio, regras no servidor e no banco, configuração por tela (parâmetros, regra fiscal, dados da instituição) — mas multi-instituição exige isolamento por tenant (ex.: `instituicao_id` + Row-Level Security no PostgreSQL), onboarding e cobrança, que **não fazem parte do MVP**.

2. Quem usa (perfis)

Perfil	O que faz	Onde trabalha
Administrador	Tudo: cadastros, usuários, perfis, parâmetros, regra fiscal, relatórios, financeiro, cancelamentos, LGPD, auditoria	Backoffice
Caixa	Abre/fecha caixa, vende, sangria/suprimento, cadastro rápido de cliente para fiado	PDV em tela cheia (entra direto nele)
Triagem	Registra lotes de doação, aprova itens (com etiqueta) e descarta com motivo	Backoffice → Entradas

Os perfis são **compostos por 24 permissões** (ex.: `caixa.operar`, `estoque.transferir`, `relatorios.consultar`), e o administrador pode criar novos. O menu só esconde o que o usuário não usaria; **quem decide é sempre o servidor**.

3. Stack tecnológica

Camada	Tecnologia	Por quê
Front-end	**React 19 + TypeScript estrito + Vite 7**, React Router 7	Interface rápida, tipagem forte ponta a ponta
PDV offline	**PWA**: Service Worker próprio + **IndexedDB**	Vender sem internet e sincronizar depois
Back-end	**Node.js ≥ 20.19 + Fastify 5 + TypeScript**	Desempenho e plugins maduros de segurança
Validação	**Zod 4**	Toda entrada validada com lista branca de campos
Banco	**PostgreSQL 16** (compatível 11+)	Regras críticas garantidas por triggers e CHECKs
ORM	**Prisma 7** com `adapter-pg`	Migrações versionadas, consultas parametrizadas
Autenticação	**JWT HS256** (`jose`) + refresh opaco em cookie HttpOnly; senhas **argon2id**	Ver seção 9
Segurança HTTP	`@fastify/helmet`, `@fastify/cors`, `@fastify/rate-limit`	CSP/HSTS, CORS restrito, limite de tentativas
Relatórios	`pdfkit` (PDF), `exceljs` (XLSX), CSV próprio	Exportação com proteção contra injeção de fórmula
Upload	`@fastify/multipart` + `file-type`	Tipo real do arquivo por *magic bytes*
Fontes	`@fontsource-variable/lora` e `/nunito`	Servidas pelo próprio app (offline + CSP `'self'`)
Monorepo	**npm workspaces**	Um `npm install`, scripts unificados

4. Estrutura do repositório

```
BAZAR/
├── apps/
│   ├── web/                 @bazar/web — PDV + Backoffice
│   │   └── src/
│   │       ├── app/         rotas e layout (menu lateral)
│   │       ├── features/    um módulo por domínio: auth, inicio, pdv, caixa, entradas,
│   │       │                estoque, financeiro, relatorios, cadastros, admin
│   │       ├── shared/      api (cliente HTTP, tipos), components, hooks, lib, offline
│   │       ├── styles/      tokens.css (brandbook) → base → components → layout → pdv → print
│   │       └── mocks/       API simulada, só em desenvolvimento (fora do build)
│   └── api/                 @bazar/api — API REST /api/v1
│       ├── prisma/          schema (33 tabelas, 18 enums), migrações 0001–0003, seed
│       ├── src/
│       │   ├── modules/     acesso, cadastros, entradas, estoque, pdv, caixa,
│       │   │                financeiro, fiscal, relatorios, admin
│       │   ├── shared/      tokens, dinheiro, datas, auditoria, documento (CPF/CNPJ)…
│       │   └── plugins/     auth.ts — AuthN/AuthZ central
│       └── tests/           smoke, concorrência, reteste-qa
├── database/                scripts SQL do DBA (DDL, papéis, testes, carga, manutenção)
└── docs/                    arquitetura, marca (brandbook), qa
```

Cada domínio tem **o mesmo nome no front e no back** (ex.: `features/financeiro` ↔ `modules/financeiro`).

5. Funcionalidades — como acontecem

5.1 Entrada e triagem de doações

Fluxo: lote recebido → triagem item a item → aprovado (vai ao estoque) ou descartado (nunca entra) → lote encerrado.

1. **Registrar o lote** (`/entradas/novo`) com tipo:
2. **Doação** — exige parceiro e/ou campanha (RN-04);
3. **Compra** — exige fornecedor e valor pago, e **gera automaticamente uma conta a pagar** (RN-05);
4. **Saldo inicial** — só pela carga inicial da implantação. Pode anexar o documento do lote (PDF/imagem; o servidor confere o tipo real).
5. **Triar** (`/entradas/:id/triagem`), em duas colunas:
6. **Aprovar:** categoria; controle **etiquetado** (peça única, ganha código `BZ000001`…) ou **por categoria** (itens comuns em quantidade); preço; **valor atribuído unitário** (doação: sugere o preço; compra: obrigatório o **custo**); estoque de destino (Bazar ou Doações).
7. **Descartar:** um registro por item, com motivo. Descartado nunca entra no estoque.
8. **Etiquetas** Code 39 impressas pelo navegador.
9. **Encerrar o lote.** Lote triado não aceita registros (RN-28); reabrir ou reverter descarte exige permissão e motivo, e vai para a auditoria.

5.2 Estoque duplo (Bazar ⇄ Doações)

- **Posição** (`/estoque`) com filtros por estoque, categoria, parceiro e campanha.
- **Histórico do item** (`/estoque/itens/:id`): linha do tempo de tudo o que aconteceu com ele.
- **Transferência** entre Bazar e Doações, com motivo e usuário registrados.
- **Baixa, ajuste e preço**: baixa por motivo, ajuste de inventário, mudança de preço da peça ou da categoria — sempre com motivo e auditoria.
- **Carga inicial** por planilha (CSV/XLSX), tudo ou nada; aceita "Doações", "doacoes", "DOAÇÕES".

Regra de ouro: ninguém escreve no saldo. Toda mudança é uma linha em `movimentacao`, e um trigger do banco atualiza o saldo. Garante: saldo nunca negativo (RN-06/07), só vende do Bazar (RN-10), histórico imutável — nada se apaga (RN-11/24).

5.3 PDV (ponto de venda)

Tela cheia, pensada para leitor de código e tablet de 10".

- **Leitura de código:** campo sempre focado; leitor USB funciona como teclado (Enter); feedback sonoro e visual.
- **Venda por categoria:** botões com preço para itens comuns, com quantidade (F2).
- **Atalhos:** F2 quantidade · F4 pagamento · Esc remove o último · F6 sangria/suprimento · F8 sincronizar.
- **Pagamento** (F4): divide entre formas (dinheiro, PIX, débito, crédito, fiado), calcula troco (só em dinheiro — RN-15), desconto até o limite do perfil.
- **Fiado:** exige cliente e vencimento; **cadastro rápido de cliente** com ciência do aviso de privacidade (LGPD); se o telefone já existe, pergunta "É a mesma pessoa?".
- **Comprovante não fiscal** com nome, CNPJ e endereço da instituição (impressão 80 mm).
- **O servidor recalcula tudo** (preço, subtotal, desconto, total, troco); a tela é só conferência.
- **PEPS** na venda por categoria, com desempate fixo (RN-09). Dois caixas disputando a última peça: só um vende (testado).

5.4 PDV offline (PWA)

1. **Sem internet a venda continua:** vai para o IndexedDB com número provisório (`OFF-XXXX-0001`) e entra na **fila**.
2. **Na reconexão** (e a cada 30 s) a fila sobe em `POST /sync/lote`. O id da venda é a chave de idempotência: reenviar não duplica. Venda recusada (ex.: peça já vendida) vira **pendência de sincronização**, resolvida pelo administrador em `/sincronizacao` (RN-26).
3. **F5 ou queda de energia sem internet:** o PDV reabre para o mesmo operador, desde que:
4. só a tela do PDV fique liberada (permissão `caixa.operar` apenas);
5. exista caixa aberto por esse operador naquele aparelho;
6. o último login online esteja dentro de `pdv.max_horas_sem_sync` (padrão 8 h);
7. nenhum token ou senha fica no aparelho, só um resumo do operador.
8. **Ao reconectar**, o sistema confirma a sessão no servidor **antes** de sincronizar. Sessão revogada → nada sobe, pede novo login, e carrinho e fila ficam guardados.
9. **Logout** bloqueado enquanto houver fila; depois apaga IndexedDB e caches. O Service Worker nunca guarda respostas da API.

5.5 Controle de caixa

Fluxo: abrir (terminal + fundo de troco) → vender / receber / sangria / suprimento → fechar (contado × esperado).

- **Um caixa aberto por terminal e um por operador** (RN-12/13, garantido no banco).
- **Sangria e suprimento** com motivo; sangria maior que o dinheiro em caixa é recusada.
- **Fechamento** (RN-19): prévia com totais por forma e **dinheiro esperado** = fundo + vendas em dinheiro + fiado recebido em dinheiro + suprimentos − sangrias. O operador informa o contado; a diferença fica registrada com justificativa. Exige fila offline vazia.
- Fiado recebido em PIX ou cartão é gravado com a forma correta; só dinheiro entra no esperado da gaveta.

5.6 Financeiro

- **Contas a pagar** (`/financeiro/pagar`): lançamento com categoria de despesa e vencimento; alertas de vencidas e a vencer na tela inicial; no pagamento, informa a forma. Se "saiu do caixa", vira sangria automática; cancelar uma conta paga com dinheiro do caixa devolve o valor como suprimento (RN-32).
- **Contas a receber / fiado** (`/financeiro/receber`): nascem das vendas fiadas ou de lançamento manual; recebimento total ou parcial, dentro ou fora do caixa, com forma obrigatória; extrato por cliente; estorno pelo administrador, sempre do recebimento inteiro e uma única vez (RN-27).
- **Resumo financeiro:** entradas por forma, fiado recebido, saídas e resultado do período.

5.7 Relatórios e prestação de contas

Todos aparecem na tela e exportam em **PDF, CSV e XLSX**, com os mesmos números e o mesmo filtro (RF-REL-07).

Relatório	O que mostra
Prestação de contas	Por **parceiro ou por campanha** (nunca somados); doação, compra e saldo inicial separados; bloco "recebido no período" (lotes, aprovados, descartados, valor atribuído), bloco "movimentado no período" (vendidos, receita, transferências, baixas) e saldo atual
Vendas	Total por dia, forma, categoria (receita com desconto rateado) ou operador, ou lista venda a venda; **canceladas sempre à parte**; imposto calculado quando houver regra tributada
Estoque	Quantidade, valor atribuído e valor de venda potencial por categoria, em cada estoque
Descartes	Filtro por parceiro e campanha; quantidade por motivo
Transferências	Cada movimento Bazar ⇄ Doações: item, sentido, quantidade, usuário e data
Fiado em aberto	Saldo por cliente, vencidas em destaque

Exemplo: "Quanto a C&A gerou em setembro?" → Prestação de contas por parceiro, período de setembro: a linha da C&A mostra itens recebidos, descartados, valor atribuído, itens vendidos no mês (inclusive de doações antigas), receita e o que ainda está em estoque.

5.8 Cadastros

Categorias, parceiros, campanhas, motivos de descarte e de baixa, categorias de despesa e clientes.

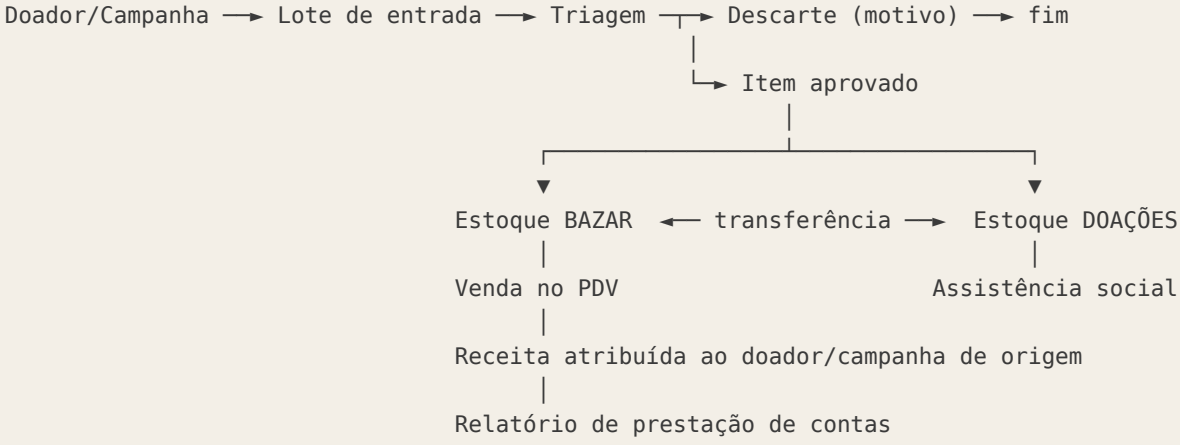
- **Nada é excluído — só desativado** (RN-24).
- Categoria vendida por categoria exige preço maior que zero.

- Parceiro com tipo (Empresa, Loja, Pessoa física, Outro) e CPF/CNPJ opcional com dígito verificador conferido. O "Doador não identificado" é especial: não pode ser desativado nem renomeado.
- Importação de cadastros por planilha, com erro por linha.

5.9 Administração, regra fiscal, auditoria e LGPD

- **Usuários e perfis:** senha provisória exibida uma única vez e troca obrigatória no primeiro acesso; perfis compostos pelas 24 permissões.
- **Terminais, parâmetros** (prazo do fiado, sessão, PDV offline…) e **dados da instituição** (aparecem no comprovante).
- **Regra fiscal versionada:** geral, por categoria ou por item; isento ou tributado, alíquota, fundamento legal e vigência. Nova regra encerra a anterior **sem alterar vendas passadas**: cada venda guarda cópia da regra usada (RN-17).
- **Auditoria imutável** de ações sensíveis (cancelamentos, ajustes, estornos, exportações), **sem dados pessoais**.
- **LGPD:** ciência do aviso de privacidade registrada; **exportação** dos dados do titular; **anonimização** irreversível (as vendas continuam nos relatórios sem identificar a pessoa); catálogo do PDV só leva clientes com fiado ativo ou recente; CPF de parceiro nunca volta inteiro da API.

6. Ciclo de vida de uma doação



Cada passo fica ligado ao lote de origem; por isso a receita de uma venda de setembro pode ser atribuída à doação recebida em março.

7. Regras de negócio essenciais e onde ficam

As regras críticas são validadas **duas vezes**: na API e no banco (defesa em profundidade).

Regra	API	Banco
Saldo nunca negativo; só vende do Bazar	trava `FOR UPDATE`	`tg_valida_movimentacao`
Peça etiquetada tem quantidade 1	validação	`ck_item_etiquetado`

Regra	API	Banco
Fiado exige cliente e conta a receber	`vendas.service.ts`	`tg_valida_venda_completa`
Pagamentos = total; troco só em dinheiro	servidor recalcula	`ck_pagamento_*`, `ck_venda_totais`
Um caixa aberto por terminal e por operador	`POST /caixas`	índices únicos parciais
Cancelamento com estorno automático	reembolso por sangria	`tg_estorna_venda_cancelada`
Nada se apaga	API sem DELETE	`fn_bloqueia_exclusao`; papel da API sem DELETE
Regra fiscal sem sobreposição de vigência	`fiscal.service.ts`	`ux_regra_vigente`

8. API

- **Base:** `/api/v1`, cerca de 80 rotas REST.
- **Erros** no padrão RFC 9457 (`application/problem+json`), com `code` de negócio (ex.: `SALDO_INSUFICIENTE`), erros por campo e `requestId`; **nunca** stack trace.
- **Toda rota declara a permissão exigida**; se alguma não declarar, a API não sobe (*fail secure*).
- **Idempotência:** venda, transferência e recebimento usam UUID gerado no cliente; reenviar não duplica.
- **Exemplos:** `PUT /vendas/:id`, `POST /sync/lote`, `GET /pdv/catalogo` (ETag), `POST /caixas`, `POST /caixas/:id/fechar`, `POST /lotes`, `POST /lotes/:id/itens`, `GET /relatorios/:tipo?formato=pdf|csv|xlsx`, `POST /titulares/:tipo/:id/anonimizar`.

9. Segurança (baseline OWASP ASVS nível 2)

Controle	Implementação
Autenticação	**JWT HS256 com algoritmo fixado**, expiração de 15 min, `iss` / `aud` validados; `alg=none` e assinatura adulterada recusados (testado)
Sessão	Refresh opaco de 256 bits em cookie `HttpOnly; Secure; SameSite=Strict`, só o hash no banco; **rotação com detecção de reuso** (revoga todas as sessões)
Senhas	argon2id; bloqueio de 10 min após 5 erros; rate limit em `/auth/*`; mensagem genérica
Autorização	No servidor, em todas as rotas; anti-IDOR (caixa/venda de outro operador → 403/404)
Entrada	Zod com lista branca (sem *mass assignment*); SQL sempre parametrizado
Upload	Tamanho limitado, tipo real conferido, nome aleatório, anti *path traversal*
HTTP	Helmet (CSP, HSTS, nosniff, `frame-ancestors 'none'`), CORS por lista (nunca `*`), `Cache-Control: no-store`
Segredos	Só por variável de ambiente, validadas na subida; produção via Azure Key Vault / Google Secret Manager; `.env*` fora do Git

Controle	Implementação
Banco	Papel `bazar_api` sem DELETE e sem escrita no saldo; `bazar_relatorio` só leitura e sem PII
Logs	pino com `redact`: sem token, cookie ou dado pessoal
Front	Token só em memória; sem PII em URL (busca de cliente no corpo do POST); telefone mascarado nas listas
Dependências	`npm audit` com 0 vulnerabilidades

10. Identidade visual

Segue o brandbook da instituição (`docs/marca/`):

- **Cores:** verde-esperança `#5B8C3E`, dourado-terra `#8C6D46`, branco-osso `#FAF7F2`, cinza-texto `#4A4A4A`; menu, cabeçalho do PDV e login em verde-floresta `#2D461F`. Onde a cor da marca não atinge WCAG AA, usa-se o mesmo tom mais escuro (ex.: botões `#4E7A34`).
- **Acentos das pétalas** identificam cada área do menu (coral: operação; laranja: estoque; amarelo: financeiro; azul: gestão).
- **Fontes: Lora** nos títulos, **Nunito** no corpo.
- Tema escuro automático derivado do verde; fonte base de 17 px, alvos de toque ≥ 44 px, foco sempre visível.

11. Qualidade — testes e verificações

Bateria	Resultado atual
`tests/smoke.ts` (fluxos críticos e segurança)	114 ok
`tests/concorrencia.ts` (disputa da última peça, IDOR)	OK
`tests/reteste-qa.ts` (regressão dos 12 tickets)	37 ok
`database/04_testes_0003.sql` (tentativas de violar regras do banco)	0 falha
`database/07_manutencao_rotina.sql` (conferência de saldo e prestação de contas)	0 divergência
E2E no navegador (PWA offline, comprovante, cadastros, relatórios)	OK
Typecheck web + api e `npm audit`	0 erros, 0 vulnerabilidades

Teste de volume (`05_carga_volume_teste.sql`): cerca de 5 anos simulados — 64 mil itens, 129 mil movimentações, 60 mil vendas.

12. Como rodar

```
npm install                                  # na raiz: instala web + api

# Só o front, com API simulada (sem banco)
cp apps/web/.env.example apps/web/.env.local # VITE_USE_MOCK=true
npm run dev:web                              # http://localhost:5173

# API real (PostgreSQL 16)
cp apps/api/.env.example apps/api/.env       # preencher segredos — nunca versionar
npm run db:deploy && npm run db:seed         # migrações + admin inicial (troca de senha
obrigatória)
psql -d bazar -v senha_api="..." -f database/03_seguranca_papeis.sql
npm run dev:api                              # http://localhost:3000/api/v1

# Testes, em banco de TESTE, nesta ordem:
npm run test:smoke && npm run test:concorrencia && npm run test:reteste
```

Produção: atrás de proxy com TLS (`TRUST_PROXY=true`), segredos vindos do cofre, pasta de uploads em volume com backup, `03_seguranca_papeis.sql` após cada deploy e `07_manutencao_rotina.sql` diariamente.

13. Status e próximos passos

Veredito do QA (2º reteste): aprovado.

Item	Situação
Ticket 13 — CNPJ alfanumérico (formato da Receita Federal a partir de julho/2026)	Recomendado antes do go-live; hoje um CNPJ com letras é recusado. Não bloqueia (documento é opcional)
PIN local no PDV offline (terminal compartilhado)	Avaliar após o piloto
RF-PDV-06 — desconto acima do limite com autorização	Depende da PA-08
RF-PDV-11 — devolução/troca	Depende da PA-14
Impressão térmica direta	Hoje usa a impressão do navegador (RNF-11)
Multi-instituição (SaaS)	Fora do MVP (ver seção 1)

Roteiro do piloto:

1. Carga inicial do estoque.
2. Cadastro de parceiros, campanhas e usuários.
3. 5 dias úteis de operação real, com conferência diária do caixa.
4. Relatório de prestação de contas por parceiro ao final.